

SHACL Explainer Tool

Feature, Scope, Pipeline, and Evaluation Report

Prepared for thesis discussion and supervisor review

Core result

The tool converts raw SHACL validation results involving `sh:node` references into explanation trees that expose the concrete leaf failures users must repair.

1. Goal And Scope

The tool is designed to explain SHACL Core validation failures that involve referencing constraints, primarily `sh:node`. Instead of leaving the user at a high-level `NodeConstraintComponent` result, it reveals the concrete property constraints inside referenced shapes that caused the top-level failure.

Item	Result / Description
In scope	SHACL Core constraints, <code>sh:node</code> referencing, nested references, property-level <code>sh:node</code> , multiple referenced-shape failures on the same focus node, diamond references, reports with or without <code>sh:detail</code> .
Out of scope	SPARQL constraints and full support for all SHACL logical constructs such as <code>sh:and</code> , <code>sh:or</code> , <code>sh:not</code> , <code>sh:xone</code> , and qualified value shapes.
Primary user question	Which exact path, value, component, and referenced-shape chain explains this validation failure?

Goal coverage

- Identifies exact failed property constraints such as `minCount`, `datatype`, `pattern`, `in`, and `maxCount`.
- Shows `sh:path` and offending `sh:value` where SHACL provides a value.
- Preserves the chain of referenced shapes through nested `sh:node` constraints.
- Aims to find all reachable leaf failures so users can fully repair the data.
- Deduplicates repeated leaf failures while preserving alternate reference paths.

2. Pipeline

The pipeline is intentionally small and reproducible. It accepts a data graph and a shapes graph, runs pySHACL, and then transforms the validation report graph into a user-facing explanation tree.

```
data.ttl + shapes.ttl
-> pyshacl.validate(...)
-> report_graph
-> get top-level sh:result entries
-> expand NodeConstraintComponent results recursively
-> build ReferenceNode / LeafFailure tree
-> deduplicate repeated leaves, preserve alternate paths
-> render as text, JSON, CSV, or summary
```

Item	Result / Description
Root nodes	Top-level referencing failures such as NodeConstraintComponent, or direct leaves if pySHACL reports them as top-level failures.
Internal nodes	Additional referencing failures discovered through sh:detail or fallback revalidation.
Leaf nodes	Concrete SHACL Core failures such as minCount, maxCount, datatype, pattern, class, or in.
Fallback path	If sh:detail is absent, the tool revalidates the focus node against the referenced shape to recover missing leaf failures.

Pipeline by source file

Item	Result / Description
1. cli.py	Input: data path, shapes path, optional external report, CLI flags. Processing: parses RDF graphs, runs pySHACL or loads a report, applies filters and output options. Output: validation report, rendered terminal output, and optional CSV/report/timing files.
2. tree.py	Input: internal design requirements. Processing: defines the explanation-tree dataclasses. Output: ReferenceNode for referencing failures and LeafFailure for concrete SHACL failures.
3. parser.py	Input: SHACL report graph. Processing: extracts only direct sh:result entries from the ValidationReport node. Output: top-level ValidationResult roots for expansion.

4. expander.py	Input: top-level results, report graph, data graph, shapes graph. Processing: recursively expands sh:node results through sh:detail or fallback revalidation. Output: raw explanation tree.
5. fallback.py	Input: target focus/value node, referenced shape, data graph, shapes graph. Processing: builds a targeted shapes graph and revalidates when sh:detail is absent. Output: reconstructed validation results with concrete leaves.
6. deduplicator.py	Input: raw explanation tree. Processing: removes duplicate leaves, merges repeated reference siblings, and preserves alternate chains. Output: deduplicated explanation tree.
7. renderer.py	Input: deduplicated explanation tree and output/filter flags. Processing: shortens prefixes, filters, summarizes, formats, and generates repair hints. Output: text tree, JSON, CSV, or summary.
8. cli.py	Input: rendered result plus timing/statistics. Processing: prints output and writes optional artifacts. Output: user-facing terminal result, saved report, CSV export, or timing CSV.

Compact module input/output view

Item	Result / Description
cli.py	Input: data graph, shapes graph, optional external report, CLI options. Output: validation report and final user-facing output.
tree.py	Input: conceptual explanation-tree structure. Output: shared ReferenceNode and LeafFailure data model.
parser.py	Input: SHACL validation report graph. Output: root validation results.
expander.py	Input: root results plus report/data/shapes graphs. Output: explanation tree.
fallback.py	Input: focus or value node and referenced shape. Output: reconstructed nested validation results.
deduplicator.py	Input: explanation tree. Output: deduplicated tree with alternate chains preserved.
renderer.py	Input: deduplicated tree. Output: text, JSON, CSV, summary, and repair hints.

3. Implemented Features

Item	Result / Description
Text tree output	Readable explanation tree with focus node, reference path, referenced shape, leaf path, component, value, message, and alternate paths.
JSON output	Machine-readable representation of reference and leaf nodes.
CSV export	Flat leaf-failure export with focus node, reference chain, path, component, value, message, kind, and repair hint.
Summary mode	Counts roots, reference nodes, leaf failures, direct vs referenced failures, triples, validation results, top paths, components, and reference paths.
Filtering	Filter by focus node, leaf path, component, or reference path.
Output limiting	Limit displayed explanation roots for large reports.
Repair hints	Actionable hints for common Core constraints such as minCount, maxCount, datatype, pattern, class, and in.
Raw report export	Save the pySHACL validation report graph as Turtle for comparison and audit.
Timing CSV	Export parse, validation, tree-building, rendering, triples, and result counts for reproducibility.
Dynamic prefixes	Uses prefixes from loaded RDF graphs, with built-in support for ex, ub, sh, and xsd.

Apache Jena compatibility

Item	Result / Description
External report input	The CLI can load an existing SHACL validation report with --report instead of always running pySHACL. This allows reports produced by Apache Jena SHACL to be explained.
No sh:detail fallback	Jena reports may not include pySHACL-style nested sh:detail results. The fallback path revalidates the relevant node against the referenced shape to recover concrete leaf failures.
Node-shape sourceShape	Jena-style reports may set sh:sourceShape to the enclosing node shape while sh:resultPath identifies the property constraint. The expander uses the original shapes graph to find the matching property shape and its sh:node target.

Property-level sh:node	When a property-level sh:node fails, the referenced shape applies to sh:value, not necessarily to sh:focusNode. The pipeline revalidates the value node so the explanation points to the real broken resource.
Supported formats	The report parser accepts RDF formats through --report-format, for example turtle for .ttl reports or xml for RDF/XML reports.
Jena fixture	test_cases/jena_report_tc7_node_source_no_detail.ttl simulates a Jena-style report for the property-level worksFor -> CompanyShape case.

Command examples

```
python3 -m shacl_explainer.cli data.ttl shapes.ttl
python3 -m shacl_explainer.cli data.ttl shapes.ttl --summary --top 5
python3 -m shacl_explainer.cli data.ttl shapes.ttl --limit 20 --hints
python3 -m shacl_explainer.cli data.ttl shapes.ttl --path ub:name
python3 -m shacl_explainer.cli data.ttl shapes.ttl --reference-path
ub:doctoralDegreeFrom
python3 -m shacl_explainer.cli data.ttl shapes.ttl --csv failures.csv --timing-csv
timings.csv
```

Apache Jena command examples

```
python3 -m shacl_explainer.cli data.ttl shapes.ttl \
--report jena_report.ttl --report-format turtle

python3 -m shacl_explainer.cli data.ttl shapes.ttl \
--report jena_report.rdf --report-format xml

python3 -m shacl_explainer.cli test_cases/tc7_property_node.ttl \
test_cases/tc7_property_node.ttl \
--report test_cases/jena_report_tc7_node_source_no_detail.ttl --hints
```

4. Output Formats

Item	Result / Description
Text tree	Best for human inspection and thesis examples.
Summary	Best for large datasets and evaluation tables.
JSON	Best for downstream tooling or UI integration.
CSV	Best for spreadsheet analysis and quantitative thesis results.
Raw report TTL	Best for comparing the explainer with the original pySHACL report.
Timing CSV	Best for reproducibility and performance evaluation.

Example text tree

```
via ex:CompanyShape path=ex:worksFor focus=ex:alice
  [minCount] path=ex:legalName
    -> ex:legalName is required
    repair: Add at least one ex:legalName value to ex:alice.
```

5. Test Coverage

The tool is covered by automated unittest tests. At the time of this report, the suite contains 24 tests and passes successfully.

Item	Result / Description
TC1	Single sh:node reference with one hidden leaf failure.
TC2	Multiple leaf failures under one referenced shape.
TC3	Two-level reference chain: ContractorShape -> EmployeeShape -> PersonShape.
TC4	Multiple sh:node branches on the same focus node.
TC5	Diamond reference with deduplication and alternate-path preservation.
TC6	Complex organization case with deeper nesting, sibling references, all failures through sh:node, and diamond reuse.
TC7	Property-level sh:node, matching the pattern discovered in the supervisor LUBM schema.
Fallback tests	Remove all sh:detail triples and verify that revalidation reconstructs the missing leaves.

Test case files

The .ttl files below form the controlled evidence base for the thesis. Each file isolates one behavior that the explanation pipeline should handle, from simple hidden leaves to Jena-style external reports.

Item	Result / Description
testdata.ttl	Small valid baseline data graph used to confirm that the CLI correctly reports conforming data.
testshapes.ttl	Companion baseline shapes graph for testdata.ttl; useful for checking the no-violation path.
tc1_single_leaf.ttl	Minimal sh:node case where one concrete datatype failure is hidden behind a referencing result.
tc2_multi_leaf.ttl	One referenced shape produces several concrete leaf failures, testing complete leaf recovery.
tc3_two_level.ttl	Nested reference chain, ContractorShape -> EmployeeShape, used to verify multi-level expansion.
tc4_mixed.ttl	Uses multiple sh:node branches to collect several referenced-shape failures on the same data node.
tc5_diamond.ttl	Diamond-shaped reference graph that tests deduplication and preservation of alternate reference chains.
tc6_complex_org.ttl	Larger organization example with deeper nesting, sibling references, all failures through sh:node, and repeated shapes.
tc7_property_node.ttl	Property-level sh:node case where the referenced value node must be revalidated, matching the LUBM pattern.
tc8_cycle_property_paths.ttl	Pathological cyclic reference case used to prove that the cycle guard stops repeated expansion.
external_report_tc2_no_detail.ttl	External report fixture with sh:detail removed, testing fallback reconstruction for TC2.
external_report_tc7_property_no_detail.ttl	External report fixture for property-level sh:node without sh:detail, testing value-node fallback.
jena_report_tc7_node_source_no_detail.ttl	Apache Jena-style report where sourceShape points to the node shape and resultPath identifies the property shape.

```
python3 -m unittest discover -s tests -v
```

```
Ran 24 tests
```

```
OK
```

6. Detailed Stress-Test Rationale

The following three tests were added to exercise risks that appear in realistic SHACL deployments: large reusable shape graphs, property-level references, and cyclic references.

TC6: tc6_complex_org.ttl

Item	Result / Description
What it models	A ProjectLead must satisfy ProjectLeadShape, which references EmploymentShape, SecurityClearanceShape, and ProjectAssignmentShape. Those shapes further reference PersonShape and ContactShape.
Potential issue	A high-level sh:node failure may hide the actual repair actions across several shapes, such as missing name, invalid age, invalid email, missing employeeId, invalid clearanceLevel, or missing projectCode.
Why this test exists	It checks practical completeness: nested references, sibling references, all concrete failures reached through sh:node, and diamond reuse of PersonShape.
Pipeline solution	parser.py selects roots, expander.py recursively expands sh:node details, deduplicator.py removes repeated PersonShape leaves while preserving alternate chains, and renderer.py displays the actionable tree.
Applicability	Useful for enterprise or ontology validation where one entity, such as employee, project lead, patient, contract, or professor, is validated through reusable referenced shapes.

TC7: tc7_property_node.ttl

Item	Result / Description
What it models	An Employee has ex:worksFor ex:acme. The worksFor value must satisfy CompanyShape.
Potential issue	The failing node is the property value ex:acme, not the original focus node ex:alice. Revalidating the wrong node would produce the wrong explanation.
Why this test exists	It tests property-level sh:node, the same pattern found in the supervisor LUBM schema where degree properties point to UniversityShape.

Pipeline solution	expander.py detects the property path and uses sh:value as the target for fallback revalidation. The leaf failure becomes ex:legalName minCount on ex:acme.
Applicability	Applies whenever a property value must satisfy another shape: worksFor company, degreeFrom university, order customer, invoice address, patient doctor, and similar object references.

TC8: tc8_cycle_property_paths.ttl

Item	Result / Description
What it models	CycleShape references itself through ex:left and ex:right, while ex:a points back to itself on both properties.
Potential issue	Without a cycle guard, expansion could loop forever: CycleShape -> CycleShape -> CycleShape, especially because the same shape is reachable through more than one property path.
Why this test exists	It proves the explainer is safe on recursive schemas and pathological inputs, not only clean acyclic examples.
Pipeline solution	expander.py keeps a visited set per traversal branch using the focus node and shape. Once ex:a against CycleShape has been expanded in that branch, recursion stops while the concrete ex:required minCount leaf remains visible.
Applicability	Applies to recursive models such as organization hierarchies, category trees, folder structures, graph nodes, dependencies, and part-whole relationships.

7. Large Dataset Evaluation: LUBM

The tool was tested on the supervisor-provided LUBM-style dataset and schema: lubm_skg_1.ttl and schema1.ttl. The data file is about 163 MB. The schema includes property-level sh:node constraints from FullProfessorShape to UniversityShape.

Item	Result / Description
Data file	/Users/mariyakezdekbayeva/Downloads/lubm_skg_1.ttl
Shapes file	/Users/mariyakezdekbayeva/Downloads/schema1.ttl
Explanation roots	891
Reference nodes	554
Leaf failures	992
Leaf path	ub:name: 992
Leaf component	minCount: 992
Reference paths	ub:doctoralDegreeFrom -> ub:UniversityShape: 307; ub:mastersDegreeFrom -> ub:UniversityShape: 161; ub:undergraduateDegreeFrom -> ub:UniversityShape: 86.

Performance

Item	Result / Description
Parse data	19.248 seconds
Parse shapes	0.001 seconds
Validate	0.811 seconds
Build explanation tree	0.070 seconds
Render summary	0.006 seconds
Interpretation	The expensive step is loading the 163 MB Turtle data graph; the explanation algorithm itself is fast on this report.

LUBM interpretation

The summary shows that the concrete repair action is consistently missing `ub:name` values. Many failures are reached through professor degree properties that point to university resources. The explainer therefore converts a broad `FullProfessor` validation failure into actionable repairs on referenced university nodes.

8. Why The Tool Works As Expected

Item	Result / Description
Completeness within scope	The controlled tests show all reachable leaf failures for single, multiple, nested, mixed, diamond, property-level, and fallback cases.
Usability	The tree output answers where the failure happened, which value/path is involved, and how it was reached.
Reproducibility	Automated tests, timing CSV, raw report export, and summary mode support repeatable evaluation.
Scalability evidence	The LUBM run demonstrates behavior on a large real dataset; tree construction and rendering are small compared with RDF parsing.
Honest limitation	The implementation focuses on sh:node-based SHACL Core references; other logical SHACL Core constructs are future work.

9. Recommended Presentation Points

- Start with TC2 or TC3 to show the explanation tree on a simple case.
- Show TC5 or TC6 to demonstrate deduplication and multi-level references.
- Show TC7 to prove property-level sh:node support, which appears in the supervisor schema.
- Use the LUBM summary table to demonstrate practical scale and quantitative evidence.
- Emphasize that the tool does not replace pySHACL; it turns pySHACL's report into actionable root-cause explanations.